

AtMem Memory Fabric

Homa inspired low latency scheduling for distributed memory retrieval, storage and context injection

Implementation specification and delivery plan

Grounded in AtMem 2.3.3 and AtBot 0.1.0

18 September 2026

Decision in one sentence. Keep AtMem as the deterministic authority. Add a memory fabric that schedules memory messages by lane, deadline, size, budget and fairness. Keep AtBot outside the hot path as an observer that proposes bounded policy changes.

This document is a planning specification. Performance numbers are targets for experiments, not release claims.

1. Executive summary

AtMem already separates memory authority from memory intelligence. AtMem owns authorization, storage, scoping, injection, correction and deletion. AtBot proposes and ranks. The current stable release is local and single writer. The next scaling problem is therefore not to move authority into an agent. It is to add a data plane that can move governed memory efficiently across storage nodes and thousands of concurrent agent requests.

The proposed Memory Fabric borrows ideas from Homa without implementing Homa itself. Homa is message oriented, receiver driven and priority aware. It focuses on short message tail latency under load. AtMem can apply similar ideas at the application layer while continuing to use ordinary network transports initially. The first release should use explicit message envelopes, two lanes, a deterministic scheduler, receiver credits, chunking and cancellation. Transport can remain TCP based at first, with QUIC or a purpose built transport evaluated later.

Principle	AtMem decision
Authority	AtMem continues to decide what may be stored, retrieved, injected, changed or deleted.
Data plane	A new Memory Fabric moves governed memory work between the authority plane and storage nodes.
Scheduling	Interactive recall and context injection are not FIFO. They are ordered by deadline, size, class, age and fairness.
AtBot	AtBot reads telemetry and proposes tuning. It does not dispatch work or bypass policy.
Dashboard	Operators see latency, queue depth, deadline misses, node health, active policy and AtBot recommendations.
Rollout	Start with two lanes and deterministic rules. Add complexity only when measurements justify it.

Primary outcome. Reduce p99 time to governed context under bursty mixed workloads without weakening provenance, scoping, deletion, evidence or fairness.

2. Current AtMem baseline

The design must extend the existing product boundary rather than create a second memory authority. The repository currently describes AtMem 2.3.3 as a host neutral Agent Black Box and reversible memory control plane. AtBot 0.1.0 is the intelligence companion. AtBot proposes and ranks; AtMem authorizes and stores. Native AtMem retrieval is the default. Delegated context mode verifies a provider decision and records delivery evidence. The dashboard is the shared operator surface.

Existing area	Current boundary	Implication for Memory Fabric
Control plane	Shadow and active modes, verification, restore and host adapter guarantees.	Fabric must sit below authorization. It must not change control mode semantics.
AtBot	Model backed inference, query expansion, reranking and bounded maintenance proposals.	Use AtBot as advisor and anomaly detector, not hot path scheduler.
Delegation	Exact scope matching, authenticated	Reuse deadline and binding concepts in

	requests, deadlines and signed results.	Fabric envelopes.
Storage	Portable local Home, canonical memory plus rebuildable derived indexes.	Distributed storage is a new capability. Canonical versus derived state must remain explicit.
Dashboard	Single operator surface with evidence, actions and progressive disclosure.	Add Memory Fabric as operational insight, not as a competing dashboard.
Current scaling boundary	Local single writer; no cloud sync or multi primary replication.	Do not present the Fabric project as distributed canonical replication until that work exists.

3. Problem statement and target workload

A distributed AtMem deployment introduces a mixed workload. A host may need a few small records immediately for model context while the same system is indexing large archives, moving artifacts, repairing indexes or writing many background memories. A single FIFO stream lets large work delay small work. Multiple independent TCP streams can reduce head of line effects but do not create a system wide scheduling policy.

- Interactive recall: small responses with tight latency budgets and direct effect on model start time.
- Context injection: governed bytes must arrive before the model boundary and may have a hard turn deadline.
- Foreground writes: small memory updates that should complete promptly but normally do not outrank an active recall.
- Bulk work: archive import, reindex, compaction, migration, large artifact movement and repair.
- Fan out: one request may query several shards or indexes and then merge the result.
- Burst: hundreds or thousands of agents can request context at nearly the same time.

Design pressure. The system must optimize message completion, not just socket throughput. A 4 KB recall that waits behind a 20 MB transfer can be a user visible failure even when total bandwidth is high.

4. Proposed architecture

Proposed AtMem Memory Fabric

Authority stays deterministic. Scheduling becomes message aware. AtBot observes and advises.

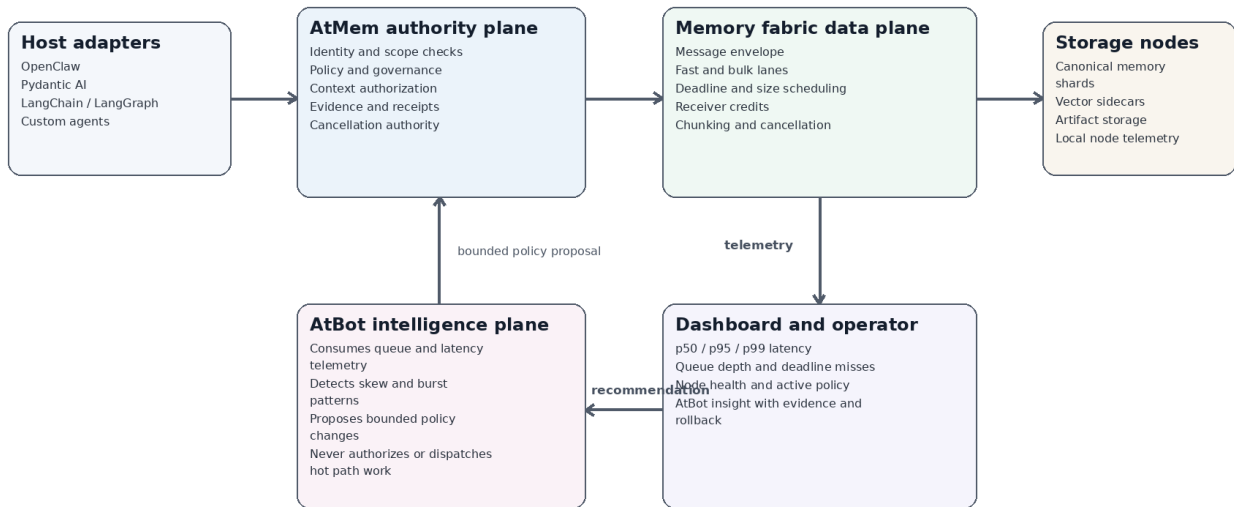


Figure 1. Proposed planes. AtBot consumes telemetry and proposes policy. AtMem remains the authority.

The architecture adds one new runtime layer and keeps the existing trust boundary. The Memory Fabric is a deterministic data plane. It accepts work only after AtMem has established identity, scope and allowed operation. It may reorder eligible work, split it into chunks, grant capacity, cancel stale work and select nodes. It may not broaden scope or authorize content.

Plane	Owns	Must not own
Authority plane	Identity, policy, scope, authorization, evidence, deletion, restore semantics.	Queue optimization based on a model guess.
Memory Fabric data plane	Envelope, lanes, scheduling, credits, chunking, cancellation, routing telemetry.	Memory truth or user permission.
AtBot intelligence plane	Analysis, queue insight, parameter proposals, anomaly explanation.	Direct dispatch, admission or canonical state.
Storage node plane	Requested read or write execution plus local load telemetry.	Cross tenant policy decisions.
Operator plane	Review, rollout, rollback and policy approval.	Hidden automatic changes without evidence.

5. Homa ideas to borrow, and what not to copy

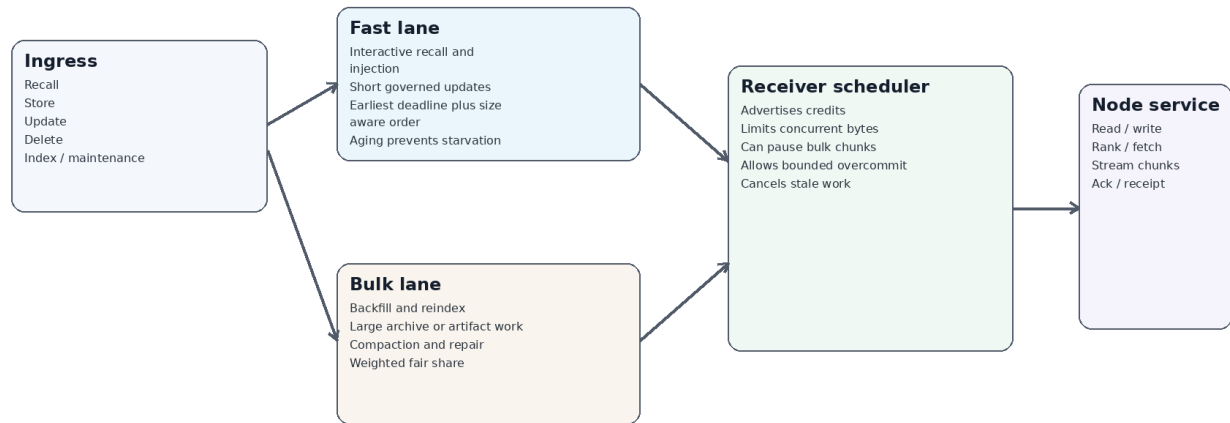
Homa was designed for low latency datacenter messages. Its design is message oriented, uses receiver managed priorities, receiver driven flow control and controlled overcommitment. The original work reports strong short message tail latency in its experimental environment. The useful part for AtMem is the scheduling model: know the work as a message, prioritize short urgent work, let the receiver advertise what it can accept, and measure tail completion time. AtMem does not need to replace the kernel transport to test these ideas.

Homa concept	AtMem adaptation
Message orientation	Every Fabric request has a complete envelope with operation, estimated bytes, deadline, scope and trace identity.
Short work first	Fast lane favors small interactive recall when deadlines are comparable.
Receiver driven control	A storage or injection receiver advertises byte and concurrency credits.
Dynamic priorities	Priority is computed at admission and can be updated when deadline, age or receiver pressure changes.
Controlled overcommitment	Permit a small number of concurrent senders or shards to keep a receiver busy, but cap total outstanding bytes.
Tail latency as primary metric	Benchmark p95 and p99 end to end recall and queue wait, not only aggregate throughput.

Do not copy packet level mechanisms into the first AtMem release. Do not claim a Homa implementation. Do not make the protocol depend on switch priority queues. The first goal is an application level scheduler that can run over current transports and produce evidence that a lower level transport change is worth the cost.

6. Lane design and queue semantics

Fast lane and bulk lane scheduling



Initial release: exactly two lanes. Add a third lane only after telemetry shows a repeatable need.

Figure 2. Two lane starting point with receiver credits and deterministic scheduling.

Start with exactly two lanes. This keeps observability and testing manageable. A third lane should be introduced only after production traces show that one class repeatedly interferes with another and cannot be fixed with weights or thresholds.

Lane	Typical work	Initial behavior
Fast	Interactive recall, context fetch, small governed updates, task state fetch.	Earliest deadline and size aware order. Tight concurrency. Age boost prevents starvation.
Bulk	Backfill, reindex, migration, large artifacts, repair, compaction.	Weighted fair service. Chunkable and preemptible between chunks. Uses spare capacity when fast lane pressure rises.

6.1 Admission rules

1. AtMem validates identity, user, agent, workspace and operation authority before Fabric admission.
2. The request is assigned a lane from deterministic operation and size rules.
3. A deadline and estimated service size are attached. Missing estimates use conservative defaults.
4. The scheduler places the message in a per receiver queue and records enqueue time.
5. A receiver grants bytes or slots. Only granted work becomes active.
6. Cancellation from the host, authority plane or expired deadline removes queued work and asks active workers to stop at the next safe boundary.

6.2 Initial lane classification

Use configuration rather than hard coded thresholds. A reasonable experiment starts with fast lane eligibility for interactive recall and injection work up to 64 KB estimated response size or up to 32 records, whichever is reached first. These values are experiment defaults, not product promises. Background operations remain bulk regardless of size unless an explicit operator policy says otherwise.

7. Message envelope and deterministic scheduler

Field	Purpose
request_id / trace_id	Idempotency, tracing and evidence correlation.
operation	recall, inject_prepare, store, update, delete, index, repair, artifact_fetch and future operations.
subject_id, user_id, agent_id, workspace_id	Existing governance identity and scope.
task_id	Optional governed task binding.
lane	fast or bulk.
deadline_at	Hard or soft completion boundary.
estimated_bytes / estimated_items	Scheduling size estimate.
token_budget	Upper bound for context delivery when relevant.
relevance_hint	Deterministic retrieval score already produced by authorized retrieval, not a new authorization signal.
can_chunk / can_cancel	Execution capabilities.
provenance_requirement	Evidence level required before delivery.
created_at / enqueue_at	Age and queue delay measurement.

The scheduler should be deterministic for the same observable state. AtBot can propose policy weights, but the hot path must not call a language model. A first scoring model can be a lexicographic policy rather than a fragile formula: lane, expired or live deadline, earliest deadline, estimated remaining size, age boost and tenant fairness deficit. This is easier to explain and test than a learned score.

Rule. Relevance can influence ordering among already authorized candidates. It must never turn an unauthorized memory into an eligible message.

8. Receiver credits, chunking and cancellation

The receiver is the component that knows its current bottleneck. A storage node knows its queue, disk or database pressure and active requests. A context injection boundary knows the current turn budget and whether the model call has already started. Therefore the receiver should advertise capacity rather than let all senders push unlimited work.

Mechanism	Initial implementation
Byte credits	Receiver advertises max outstanding bytes by lane. Credits refill when work completes or is cancelled.
Concurrency credits	Cap active messages per lane and per node. Keep a separate cap for expensive operations.
Chunking	Large bulk messages execute in bounded chunks so fast work can enter between chunks.
Cancellation	Queued work is removed immediately. Active chunkable work stops at next safe chunk boundary.
Deadline expiry	Expired recall is withheld rather than injected late. Record

	the reason in evidence.
Overcommit	Allow a small bounded excess of senders when measurements show receiver idle gaps. Never allow unbounded outstanding bytes.

A practical first implementation can model a grant as application credits rather than packet grants. For example, a node may expose 8 fast message slots, 2 bulk slots, 256 KB fast outstanding bytes and 8 MB bulk outstanding bytes. The exact values belong in policy and must be tuned from benchmarks.

9. AtBot as queue advisor

AtBot is well placed to reason about the system because it already acts as AtMem intelligence companion. Its role here should remain advisory. It receives aggregated telemetry after the fact or on a slow control loop. It can detect patterns and propose a bounded configuration change. AtMem validates the proposal against policy before any operator or automated rollout.

AtBot input	Possible insight	Allowed output
p99 fast queue wait rising while bulk bytes are high	Bulk is occupying too much receiver capacity.	Suggest lower bulk credit or smaller chunk size.
Fast queue contains many 50 to 200 KB messages	Current fast threshold is too broad.	Suggest lower fast size threshold.
Repeated short deadline misses with idle receiver windows	Grant refill or concurrency is too conservative.	Suggest a small increase within configured bounds.
One tenant dominates fast service	Fairness deficit is not correcting quickly enough.	Suggest stronger tenant weight or cap.
One shard has higher service time	Shard skew or node problem.	Flag node and recommend rebalance investigation.
Oscillating policy changes	Control loop is unstable.	Recommend freeze and rollback to last known good policy.

9.1 Recommendation contract

Every recommendation should contain: observation window, affected scope, current value, proposed value, reason, supporting metrics, expected effect, confidence, safety bounds, expiry time and rollback value. The initial release should require explicit operator apply. Later automation may apply only changes that stay inside preapproved bounds and pass a canary.

Hot path boundary. AtBot must never be required for a recall to complete. If AtBot is unavailable, the Fabric continues with the last valid deterministic policy.

10. Dashboard additions

The dashboard should expose Memory Fabric as an operational view and as evidence behind slow turns. Follow the existing design language: one clear condition, supporting facts, last check time, and at most one primary action. Avoid a second global health score.

Dashboard element	What it shows
Fabric condition	Normal, attention or degraded with last measured time.
Latency	p50, p95 and p99 end to end, queue wait and service time for fast and bulk.
Queue depth	Messages and bytes waiting by lane plus oldest message age.
Deadlines	Miss count, miss ratio and top reason categories.
Receiver credits	Current active and configured byte and concurrency limits.
Node health	Service latency, backlog, error rate and current route weight.
AtBot insight	One actionable recommendation with evidence, expected

	effect and rollback.
Policy history	Who changed what, when, why, canary result and rollback status.

A slow context event should link to its trace. The trace should show authorization time, enqueue time, wait time, grant time, node service, merge or rank time, injection preparation and final delivery or withholding. This keeps performance evidence connected to AtMem governance evidence.

11. Proposed code map

Keep the new layer explicit instead of spreading queue logic through the existing control manager. The following layout is a proposal and can be adjusted after code review.

```
atmem/fabric/
envelope.py    # typed message contract and validation
lanes.py       # lane classification and policy
scheduler.py   # deterministic ordering and fairness
credits.py     # receiver byte and concurrency grants
router.py      # storage node selection and fan out
transport.py   # TCP / HTTP initially, pluggable later
telemetry.py   # queue and service metrics
policy.py      # versioned scheduler policy
trace.py       # per request timing and evidence bridge

atmem/control/
manager.py     # admission hook after authority checks
web.py        # dashboard API additions
assets/       # Fabric cards and trace UI

packages/atbot/src/atbot/
fabric_advisor.py # telemetry analysis and bounded proposals
```

12. Delivery plan and checkpoints

Phase	Implementation tasks	Checkpoint
P0 Baseline and traces	Instrument current recall, store, ranking and injection timings. Build mixed short and bulk load generator. Capture p50, p95, p99 and CPU.	No scheduling changes. Repeatable benchmark and trace schema committed.
P1 Envelope and lanes	Add typed Fabric envelope, fast and bulk classification, policy file and no op scheduler adapter.	Existing behavior preserved when Fabric is disabled. Every request has trace identity and lane.
P2 Deterministic scheduler	Implement per receiver queues, deadline and size ordering, age boost and tenant fairness.	Fast p99 improves versus single FIFO in synthetic mixed load with no authorization regression.
P3 Receiver credits	Add byte and concurrency credits, chunk boundaries, cancellation and deadline expiry.	Bulk cannot monopolize receiver. Cancelled or expired context is never injected late.
P4 Distributed storage nodes	Add node registry, health, route selection, fan out, merge and retry rules.	Node loss has defined behavior. Evidence identifies which nodes served each request.
P5 AtBot advisor	Create aggregated telemetry feed and bounded recommendation contract.	AtBot can propose but cannot change active policy without allowed apply path.
P6 Dashboard	Add Fabric condition, lane metrics, node health, trace drill down, recommendation and policy history.	Operator can explain a slow context event without reading logs.
P7 Hardening and transport experiment	Canary policy changes, rollback, fault injection, QUIC or alternative transport A/B.	Transport change is adopted only if it improves measured tail latency after scheduler gains are isolated.

12.1 Checkpoint gates

- Safety gate: all existing memory scoping, poisoning, withholding, deletion and evidence tests remain green.
- Determinism gate: the scheduler produces the same ordering for identical policy and queue state.
- Starvation gate: no eligible bulk class can remain unserved indefinitely under sustained fast load.
- Deadline gate: expired context is never silently injected after its deadline.
- Fallback gate: disabling Fabric restores the previous local path without data migration.
- AtBot gate: removing or stopping AtBot does not break admission, scheduling, storage or recall.
- Evidence gate: every policy change and request route is traceable to an actor or deterministic rule.

13. Benchmark plan and acceptance criteria

Measure the scheduler before considering a lower level transport. The comparison must isolate each mechanism so the team can see where the gain comes from.

Profile	Workload
A Small recall	1 to 8 KB result, single shard, 1 to 1000 concurrent agents.
B Mixed	80 percent small recall, 15 percent small writes, 5 percent 256 KB to 8 MB bulk work.
C Fan out	Recall over 8, 32 and 128 shards with top k merge.

D Burst	Idle to 2000 simultaneous requests within a short window.
E Adversarial bulk	Continuous reindex or archive transfer while interactive recall runs.
F Node degradation	One slow node, one failed node and skewed shard distribution.
G Governance heavy	Same load with full evidence and policy checks enabled.

Variant	Purpose
Current AtMem local path	Baseline.
Distributed single FIFO	Shows cost of naive distribution.
Two lanes FIFO inside each lane	Measures lane separation alone.
Deadline and size scheduler	Measures scheduling gain.
Scheduler plus credits and chunking	Measures receiver control gain.
Alternative transport	Only after application layer gains are known.

Initial acceptance should be relative, not a fixed microsecond promise. A useful gate is at least 50 percent lower fast lane p99 queue wait than distributed single FIFO under the agreed mixed workload, with no more than 5 percent loss in aggregate completed bytes per second, no safety regression, bounded fairness and no late injection after deadline. These are experiment gates and should be revised after P0 establishes realistic baselines.

14. Failure modes and guardrails

Risk	Guardrail
Bulk starvation	Age boost plus minimum bulk service share.
Fast lane abuse	Operation based admission, size cap, per tenant budgets and audit.
Priority inversion	Receiver can reprioritize pending work when a tighter deadline arrives.
Bad size estimate	Track estimate error and fall back to chunking or remaining size update.
AtBot oscillation	Slow control window, bounded deltas, cool down, canary and automatic rollback on regression.
Node hot spot	Per node queue telemetry, route weighting and shard rebalance recommendation.
Retry storm	Idempotent request ID, bounded retries, jitter and circuit state.
Split brain authority	Fabric never decides authority. Canonical replication is a separate project with explicit consistency semantics.
Evidence gap	Trace admission, schedule decision, grant, node execution and final delivery separately.
Transport distraction	Do not change transport until queue and service decomposition shows transport is material.

15. Planning tasks ready for implementation

Task A: Fabric contracts

- Define envelope dataclass or Pydantic model.
- Define operation enum and lane enum.
- Define deadline semantics and cancellation state.
- Add serialization version and compatibility tests.

Task B: Scheduler core

- Implement per receiver queues.
- Implement deterministic comparator.
- Add age boost and tenant fairness accounting.
- Add simulation tests for ordering, starvation and deadline expiry.

Task C: Receiver credits

- Define credit snapshot and update API.
- Add active byte and message accounting.
- Add safe chunk boundaries for bulk operations.
- Add cancellation and credit return on failure.

Task D: Telemetry

- Record queue wait, service time, end to end time, bytes, lane, node and outcome.
- Add histograms for p50, p95 and p99.
- Add per node and per tenant dimensions with cardinality controls.

Task E: AtBot advisor

- Define read only telemetry input.
- Implement recommendation schema.
- Add rule based baseline advisor before model backed advice.
- Add bounds, expiry and rollback to every proposal.

Task F: Dashboard

- Add Fabric status API.
- Add lane metrics and node health cards.
- Add trace drill down.
- Add recommendation apply and rollback with explicit operator action.

Task G: Benchmark and release gates

- Create mixed load generator.
- Commit baseline JSON.
- Add CI invariant tests for authority and late injection.
- Publish benchmark method before publishing performance claims.

16. Definition of done for first Memory Fabric release

- Two lanes are implemented and configurable.
- All Fabric requests have versioned envelopes, trace IDs and deadlines where relevant.
- Scheduler ordering is deterministic and covered by simulation tests.
- Receiver credits bound outstanding bytes and active messages.
- Bulk work can be chunked and yields to interactive work at safe boundaries.
- Cancellation and deadline expiry cannot result in late context injection.
- AtBot recommendations are visible but cannot bypass AtMem authority.
- Dashboard shows fast and bulk p50, p95, p99, queue depth, deadline misses and node state.
- An operator can roll back a scheduler policy to the previous known value.
- Benchmark compares current path, FIFO, two lanes, scheduler, credits and any transport experiment.
- Existing AtMem safety and evidence suites remain green.
- Documentation states clearly that distributed canonical replication is not implied by the Memory Fabric.

17. Sources and design basis

AtMem repository basis

AtMem repository and README: <https://github.com/aetna000/atmem>

Current status, release 2.3.3: <https://github.com/aetna000/atmem/blob/main/docs/current-status.md>

AtBot companion boundary: <https://github.com/aetna000/atmem/blob/main/packages/atbot/README.md>

OpenClaw control plane guarantees: <https://github.com/aetna000/atmem/blob/main/docs/control-plane.md>

Delegated context authority: <https://github.com/aetna000/atmem/blob/main/docs/delegated-context-provider.md>

Data storage and backup: <https://github.com/aetna000/atmem/blob/main/docs/data-storage-and-backup.md>

Dashboard design language: <https://github.com/aetna000/atmem/blob/main/docs/dashboard-design-language.md>

Homa research basis

B. Montazeri, Y. Li, M. Alizadeh, J. Ousterhout. Homa: A Receiver Driven Low Latency Transport Protocol Using Network Priorities. SIGCOMM 2018. <https://arxiv.org/abs/1803.09615>

J. Ousterhout. A Linux Kernel Implementation of the Homa Transport Protocol. USENIX ATC 2021. <https://www.usenix.org/conference/atc21/presentation/ousterhout>

J. Ousterhout. It is Time to Replace TCP in the Datacenter. 2022. <https://arxiv.org/abs/2210.00714>

Interpretation note

Homa is used here as design inspiration for message orientation, receiver driven control, priorities, bounded overcommitment and tail latency measurement. This document does not claim that AtMem implements Homa, replaces RDMA, or will reproduce Homa paper results. Any AtMem performance claim requires the benchmark plan above on the actual deployment stack.